

Autocall Replication

Pipeline Python de pricing et de réplication d'un produit autocall à partir d'un portefeuille de vanilles européennes et binaires.

Le projet vise à étudier dans quelle mesure un payoff d'autocall, produit structuré discret à barrière et rappel anticipé, peut être approché par une combinaison linéaire d'options vanilles. Plusieurs notions de réplication sont explorées : réplication de surface de prix, réplication d'espérance de payoff, et réplication pathwise scénario par scénario.

1. Objectif du projet

L'objectif est de construire un pipeline complet permettant de :

1. définir un autocall simple mono-sous-jacent ;
2. simuler des trajectoires de sous-jacent sous modèle Black-Scholes ;
3. calculer les cashflows et le prix Monte Carlo de l'autocall ;
4. construire une base de vanilles européennes et binaires ;
5. approximer l'autocall par une combinaison linéaire de ces vanilles ;
6. mesurer la qualité de réplication selon plusieurs critères ;
7. comparer les résultats sur des grilles de spot, de date et sur des trajectoires simulées.

Le projet est encore en phase exploratoire : il s'agit d'un outil de recherche/prototypage, pas d'une librairie de pricing industrielle.

2. Description financière

Le produit étudié est un autocall discret mono-sous-jacent.

Hypothèses principales

- Sous-jacent unique.
- Modèle Black-Scholes à volatilité constante.
- Taux sans risque constant.
- Rendement continu du dividende constant.
- Dates d'observation discrètes.
- Coupon conditionnel.
- Rappel anticipé si le sous-jacent dépasse un certain seuil.
- Barrière down observée sur la trajectoire.
- Remboursement final dépendant de la barrière.

Exemple de structure

Par défaut, l'autocall possède les paramètres suivants :

```
AutocallProduct(
    notional=100.0,
    autocall_strike=1.00,
    coupon_barrier=0.80,
    coupon_rate=0.05,
    down_barrier=0.60,
    observation_months=[3, 6, 9, 12]
)
```

Cela correspond à un produit de nominal 100, observé trimestriellement, rappelé si le spot observé est strictement supérieur à 100 % du spot initial, avec coupon conditionnel de 5 % du nominal si le spot est au-dessus de 80 % du spot initial.

3. Architecture du projet

Structure recommandée du repository :

```
autocall-replication/
|
├─ products_core.py
├─ monte_carlo.py
├─ benchmark_general.py
├─ target_price.py
├─ benchmark_mean_payoff.py
├─ benchmark_payoff.py
├─ a_functional_analysis.py
|
├─ notebooks/
|   └─ notebook.ipynb
|       └─ straddle_spectral_replication.ipynb
|
├─ README.md
└─ requirements.txt
```

Remarque importante sur les noms de fichiers

Si les fichiers ont été exportés ou téléchargés avec des suffixes du type :

```
products_core(2).py
monte_carlo(2).py
benchmark_general(3).py
target_price(3).py
```

il faut les renommer avant exécution :

```
products_core.py
monte_carlo.py
benchmark_general.py
target_price.py
```

Sinon les imports Python du type :

```
from products_core import AutocallProduct
from monte_carlo import price_autocall_bs_mc
```

ne fonctionneront pas.

4. Modules principaux

4.1 products_core.py

Ce fichier contient les objets financiers de base.

Il définit notamment :

- AutocallProduct
- VanillaProduct
- EuropeanCall
- EuropeanPut
- BinaryCall
- BinaryPut

Il contient aussi les fonctions utilitaires Black-Scholes :

- norm_cdf
- norm_pdf
- _year_fraction
- bs_d1_d2

Le rôle principal de ce module est de centraliser la définition des produits financiers et leurs payoffs.

4.2 monte_carlo.py

Ce fichier contient la logique de simulation et de pricing Monte Carlo.

Fonctions principales :

```
simulate_gbm_path(...)
```

Simule une trajectoire de sous-jacent sous Black-Scholes.

```
autocall_discounted_payoff_from_path(...)
```

Calcule le payoff actualisé de l'autocall sur une trajectoire donnée.

```
price_autocall_bs_mc(...)
```

Price l'autocall par Monte Carlo en simulant plusieurs trajectoires.

4.3 `benchmark_general.py`

Ce fichier contient les outils généraux utilisés par les différents benchmarks.

Fonctions principales :

```
build_autocall_training_grid_axes(...)
```

Construit une grille de dates et de spots pour entraîner ou tester une répliation.

```
build_naive_vanilla_basis(...)
```

Construit une base naïve de vanilles : calls, puts, binaires selon le paramètre `family`.

```
fit_ridge(...)
```

Résout une régression ridge :

```
min_w ||Aw - b||2 + λ||w||2
```

```
benchmark_metrics(...)
```

Calcule les métriques d'erreur :

- MAE ;
- RMSE ;
- erreur maximale absolue ;
- biais moyen.

```
plot_benchmark_surface(...)
```

Affiche la surface cible, la surface répliquée, et l'erreur.

4.4 `target_price.py`

Ce module répond à la question suivante :

Est-ce qu'un portefeuille de vanilles peut reproduire la surface de prix actualisée de l'autocall ?

Pipeline :

1. construire une grille `(date, spot)` ;
2. calculer le prix Monte Carlo de l'autocall sur chaque point ;
3. calculer les prix Black-Scholes des vanilles sur la même grille ;
4. fitter les poids du portefeuille par ridge ;
5. comparer la surface cible et la surface répliquée.

Fonction principale :

```
run_naive_benchmark_price(...)
```

4.5 `benchmark_mean_payoff.py`

Ce module répond à la question suivante :

Est-ce qu'un portefeuille de vanilles peut reproduire l'espérance du payoff brut de l'autocall ?

Ici, on travaille sur des payoffs non actualisés.

Pipeline :

1. simuler l'espérance du payoff brut de l'autocall ;
2. calculer l'espérance du payoff brut des vanilles ;
3. fitter un portefeuille linéaire ;
4. comparer les erreurs sur train et test.

Fonction principale :

```
run_naive_benchmark_payoff(...)
```

4.6 `benchmark_payoff.py`

Ce module répond à la question suivante :

Sur une trajectoire simulée donnée, est-ce qu'un portefeuille fixe de vanilles reproduit bien le payoff actualisé de l'autocall ?

C'est le benchmark le plus proche d'une logique de réplication pathwise.

Pipeline :

1. simuler plusieurs trajectoires du sous-jacent ;
2. calculer pour chaque trajectoire le payoff actualisé de l'autocall ;
3. calculer sur la même trajectoire les payoffs actualisés des vanilles ;
4. fitter les poids du portefeuille ;
5. analyser les erreurs scénario par scénario.

Fonctions principales :

```
build_pathwise_payoff_dataset(...)  
run_naive_benchmark_payoff_pathwise(...)  
plot_pathwise_benchmark(...)
```

4.7 a_functional_analysis.py

Ce fichier contient une exploration théorique de la décomposition spectrale du noyau de straddle :

```
|x - y|
```

Il définit les fréquences, valeurs propres, coefficients et fonctions propres associées :

```
omega_n(...)  
lambda_n(...)  
c_n(...)  
phi_n(...)
```

Cette partie sert à explorer les fondements mathématiques de la réplication par straddles ou par bases fonctionnelles.

5. Installation

5.1 Cloner le repository

```
git clone <repository-url>  
cd autocall-replication
```

5.2 Créer un environnement virtuel

```
python -m venv .venv
```

Sous macOS/Linux :

```
source .venv/bin/activate
```

Sous Windows :

```
.venv\Scripts\activate
```

5.3 Installer les dépendances

Créer un fichier `requirements.txt` :

```
numpy  
pandas  
scipy  
matplotlib  
ipython  
jupyter
```

Puis installer :

```
pip install -r requirements.txt
```

6. Exemple d'utilisation

6.1 Définir un autocall

```
import pandas as pd  
  
from products_core import AutocallProduct  
  
product = AutocallProduct(  
    notional=100.0,  
    autocall_strike=1.00,  
    coupon_barrier=0.80,  
    coupon_rate=0.05,  
    down_barrier=0.60,  
    observation_months=[3, 6, 9, 12]  
)
```

```
spot0 = 100.0
valuation_date = pd.Timestamp("2025-01-01")
maturity_date = pd.Timestamp("2026-01-01")

rate = 0.03
dividend_yield = 0.00
vol = 0.20
```

6.2 Pricer l'autocall par Monte Carlo

```
from monte_carlo import price_autocall_bs_mc

res = price_autocall_bs_mc(
    product=product,
    spot0=spot0,
    valuation_date=valuation_date,
    maturity_date=maturity_date,
    rate=rate,
    dividend_yield=dividend_yield,
    vol=vol,
    n_paths=20000,
    seed=42
)

print(res["actualized_price"])
print(res["std_error"])
```

6.3 Benchmark de réplcation de surface de prix

```
from target_price import run_naive_benchmark_price
from benchmark_general import plot_benchmark_surface

out = run_naive_benchmark_price(
    product=product,
    spot0=spot0,
    valuation_date=valuation_date,
    maturity_date=maturity_date,
    rate=rate,
    dividend_yield=dividend_yield,
    vol=vol,
    family="full",
    n_spots=15,
    n_paths=3000,
    l2=1e-6,
    train_frac=0.7,
```

```

        seed=42
    )

    print(out["metrics_train"])
    print(out["metrics_test"])

    plot_benchmark_surface(
        dates=out["dates"],
        spots=out["spots"],
        target_tensor=out["target_tensor"],
        fitted_tensor=out["fitted_tensor"],
        title_prefix="Price surface - "
    )

```

6.4 Benchmark pathwise

```

from benchmark_payoff import (
    run_naive_benchmark_payoff_pathwise,
    plot_pathwise_benchmark
)

out = run_naive_benchmark_payoff_pathwise(
    product=product,
    spot0=spot0,
    valuation_date=valuation_date,
    maturity_date=maturity_date,
    rate=rate,
    dividend_yield=dividend_yield,
    vol=vol,
    family="full",
    n_paths=5000,
    l2=1e-6,
    train_frac=0.7,
    seed=42
)

print(out["metrics_train"])
print(out["metrics_test"])

plot_pathwise_benchmark(
    y=out["y"],
    replica=out["replica"],
    title_prefix="Pathwise replication - "
)

```

7. Types de bases de vanilles

La fonction `build_naive_vanilla_basis` permet de choisir plusieurs familles de produits.

```
family="calls"
```

Base composée uniquement de calls européens.

```
C(K, T)
```

```
family="calls_puts"
```

Base composée de calls et puts européens.

```
C(K, T), P(K, T)
```

```
family="full"
```

Base composée de calls, puts, binary calls et binary puts.

```
C(K, T), P(K, T), BC(K, T), BP(K, T)
```

Cette dernière base est plus riche et permet souvent une meilleure approximation des discontinuités ou effets de seuil liés aux barrières et au rappel anticipé.

8. Métriques

Les benchmarks retournent des métriques d'erreur sous forme de dictionnaire.

```
{
  "mae": ...,
  "rmse": ...,
  "max_abs_error": ...,
  "mean_error": ...
}
```

MAE

Erreur absolue moyenne.

```
MAE = mean(|réplique - cible|)
```

RMSE

Erreur quadratique moyenne.

$$\text{RMSE} = \sqrt{\text{mean}((\text{réplique} - \text{cible})^2)}$$

Max absolute error

Erreur maximale en valeur absolue.

Mean error

Erreur moyenne signée, aussi appelée biais.

9. Différence entre les benchmarks

Il est important de ne pas confondre les trois benchmarks.

9.1 Surface de prix

Dans `target_price.py`, on cherche à reproduire :

$$\text{Prix autocall}(t, S)$$

par une combinaison :

$$\sum w_i \times \text{Prix vanille}_i(t, S)$$

C'est une réplique de valeur de marché.

9.2 Espérance de payoff brut

Dans `benchmark_mean_payoff.py`, on cherche à reproduire :

$$E[\text{payoff brut autocall} \mid S_t = S]$$

par :

$$\sum w_i \times E[\text{payoff brut vanille}_i \mid S_t = S]$$

Ce benchmark ne travaille pas directement sur des prix actualisés.

9.3 Payoff pathwise

Dans `benchmark_payoff.py`, on cherche à reproduire, trajectoire par trajectoire :

Payoff actualisé autocall(ω)

par :

$\sum w_i \times \text{Payoff actualisé vanille}_i(\omega)$

C'est le benchmark le plus strict, car il teste la réplication scénario par scénario.

10. Points d'attention

10.1 La réplication est statique

Le portefeuille de vanilles est fixé une fois pour toutes.

Il ne s'agit pas encore d'une stratégie dynamique de couverture en delta, vega, gamma ou barrier risk.

10.2 Les vanilles utilisées sont synthétiques

Les options sont pricées via Black-Scholes avec volatilité constante.

Le projet ne prend pas encore en compte :

- surface de volatilité implicite ;
 - bid-ask ;
 - liquidité ;
 - smile ;
 - skew ;
 - coûts de transaction ;
 - funding ;
 - dividendes discrets ;
 - calendrier de marché réel.
-

10.3 Le modèle est Black-Scholes

Les trajectoires sont simulées sous dynamique lognormale :

$$dS_t / S_t = (r - q) dt + \sigma dW_t$$

Cela reste une hypothèse simplificatrice forte, notamment pour des produits à barrière ou rappel anticipé.

10.4 La grille de maturités doit être clarifiée

La base naïve de vanilles peut inclure plusieurs maturités intermédiaires.

Il faut distinguer :

- maturités aux dates d'observation ;
- maturités mensuelles ;
- maturités disponibles sur le marché ;
- maturités personnalisées.

Cette convention a un impact important sur la qualité de réplication.

11. Roadmap

Étape 1 — Nettoyage du repository

- Renommer les fichiers sans suffixes `(2)`, `(3)`, etc.
 - Séparer les notebooks et les modules `.py`.
 - Ajouter un vrai `requirements.txt`.
 - Ajouter un script de test rapide.
-

Étape 2 — Stabilisation du pricing

- Vérifier les conventions de dates.
 - Clarifier le calendrier utilisé : jours calendaires ou jours ouvrés.
 - Ajouter des tests unitaires sur les payoffs.
 - Vérifier les cas limites :
 - autocall immédiat ;
 - barrière touchée ;
 - barrière non touchée ;
 - coupon payé ;
 - coupon non payé ;
 - maturité atteinte sans rappel.
-

Étape 3 — Amélioration des bases de réplication

- Ajouter un choix explicite de maturités :
 - observation dates ;
 - monthly grid ;
 - custom maturities.
- Ajouter des strikes plus adaptatifs autour des barrières.
- Ajouter des digitals ou call spreads pour approximer les discontinuités.

- Ajouter contraintes de régularisation ou de sparsité.
-

Étape 4 — Robustesse statistique

- Ajouter intervalles de confiance Monte Carlo.
 - Utiliser des seeds contrôlés.
 - Ajouter variance reduction :
 - antithetic variates ;
 - control variates ;
 - common random numbers ;
 - quasi-Monte Carlo.
-

Étape 5 — Passage vers données de marché

- Importer une surface de volatilité implicite.
 - Pricer les vanilles à partir de données de marché.
 - Calibrer le modèle sur les prix observés.
 - Comparer réplication théorique et réplication réaliste.
-

Étape 6 — Réplication dynamique

- Calculer les Greeks de l'autocall.
 - Construire une couverture dynamique.
 - Comparer :
 - réplication statique par vanilles ;
 - delta hedging ;
 - hedge hybride statique + dynamique.
-

12. Limites actuelles

Le pipeline actuel est un prototype de recherche.

Les résultats ne doivent pas être interprétés comme des prix de marché définitifs ni comme une stratégie de trading directement exploitable.

Les principales limites sont :

- modèle Black-Scholes simplifié ;
- absence de smile de volatilité ;
- absence de coûts de transaction ;
- absence de liquidité et bid-ask ;
- base de vanilles synthétique ;
- conventions calendaires simplifiées ;
- absence de calibration marché ;
- réplication statique uniquement.

13. Interprétation attendue des résultats

Un bon résultat sur la surface de prix ne garantit pas nécessairement une bonne réplification pathwise.

Inversement, un fit pathwise peut réduire l'erreur scénario par scénario mais produire un portefeuille difficilement interprétable ou instable.

Il faut donc comparer simultanément :

1. les erreurs train/test ;
2. la stabilité des poids ;
3. la concentration du portefeuille ;
4. les erreurs par zone de spot ;
5. les erreurs autour des barrières ;
6. les erreurs autour des dates d'observation ;
7. la robustesse aux paramètres de marché.

14. Exemple de workflow recommandé

```
# 1. Définir le produit
product = AutocallProduct()

# 2. Pricer l'autocall par Monte Carlo
mc_price = price_autocall_bs_mc(...)

# 3. Tester une base simple de calls
res_calls = run_naive_benchmark_price(..., family="calls")

# 4. Tester calls + puts
res_calls_puts = run_naive_benchmark_price(..., family="calls_puts")

# 5. Tester base complète avec binaires
res_full = run_naive_benchmark_price(..., family="full")

# 6. Comparer les métriques
print(res_calls["metrics_test"])
print(res_calls_puts["metrics_test"])
print(res_full["metrics_test"])

# 7. Analyser les poids
print(res_full["weights"].head(20))

# 8. Tester la réplification pathwise
pathwise = run_naive_benchmark_payoff_pathwise(..., family="full")
plot_pathwise_benchmark(pathwise["y"], pathwise["replica"])
```

15. Avertissement

Ce projet est destiné à un usage pédagogique, académique et exploratoire.

Il ne constitue pas un conseil en investissement, un outil de trading validé, ni un moteur de pricing certifié.

Toute utilisation dans un cadre professionnel nécessiterait des validations supplémentaires : tests unitaires, revue quantitative, calibration marché, contrôle du risque modèle, robustesse numérique et validation indépendante.

16. Auteur

Projet développé dans le cadre d'un travail exploratoire sur la réplication d'autocalls par portefeuilles de vanilles.

17. Statut du projet

Statut actuel : prototype fonctionnel en développement.

Les prochaines priorités sont :

1. nettoyer l'architecture du repository ;
2. stabiliser les conventions de dates et de maturités ;
3. ajouter des tests unitaires ;
4. améliorer la base de réplication ;
5. intégrer progressivement des données de marché.